US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12401494
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	Mishina; Ibuki et al.

Machine learning using secure gradient descent computation

Abstract

A calculation of a gradient descent method in secure computing is performed at high speed while maintaining accuracy. A secure gradient descent computation method calculates a gradient descent method while keeping a gradient and a parameter concealed. An initialization unit initializes concealed values [M], [V] of matrices M, V (S**11**). A gradient calculation unit determines concealed value [G] of a matrix G of a gradient g (S**12**). A parameter update unit calculates [M] β1 [M]+(1−β1) [G] (S**13-1**), calculates [V]←β2 [V]+(1−β2) [G]◯[G] (S**13-2**), calculates [M{circumflex over ( )}]←β{circumflex over ( )}1, t [M] (S**13-3**), calculates [V{circumflex over ( )}]←β{circumflex over ( )}2, t [V] (S**13-4**), calculates [G{circumflex over ( )}]←Adam ([V{circumflex over ( )}]) (S**13-5**), calculates [G{circumflex over ( )}]←[G{circumflex over ( )}]◯[M{circumflex over ( )}] (S**13-6**), and calculates [W]←[W]−[G{circumflex over ( )}] (S**13-7**).

Inventors:	**Mishina; Ibuki (Musashino, JP), Ikarashi; Dai (Musashino, JP), Hamada; Koki (Musashino, JP)**
Applicant:	**NIPPON TELEGRAPH AND TELEPHONE CORPORATION** (Tokyo, JP)

Family ID:	**1000008777980**
Assignee:	**NIPPON TELEGRAPH AND TELEPHONE CORPORATION (Tokyo, JP)**
Appl. No.:	**17/634237**
Filed (or PCT Filed):	**August 14, 2019**
PCT No.:	**PCT/JP2019/031941**
PCT Pub. No.:	**WO2021/029034**
PCT Pub. Date:	February 18, 2021

Prior Publication Data

Document Identifier	Publication Date
US 20220329408 A1	Oct. 13, 2022

Publication Classification

Int. Cl.: **H04L9/06** (20060101); **G06N3/084** (20230101)

U.S. Cl.:

CPC **H04L9/06** (20130101); **G06N3/084** (20130101); H04L2209/046 (20130101)

Field of Classification Search

CPC: H04L (9/06); H04L (2209/046); H04L (2209/46); H04L (9/085); G06N (3/084)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
11816575	12/2022	Gu	N/A	G06F 21/606
2018/0137405	12/2017	Moon	N/A	G06N 3/047
2024/0281208	12/2023	Hamada	N/A	G06F 7/544

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
2018/174873	12/2017	WO	N/A

OTHER PUBLICATIONS

Kingma, et al. “ADAM: a Method for Stochastic Optimization”, Optimization, ICLR 2015, Jul. 23, 2015 (Jul. 23, 2015), XP055425253, Retrieved from the Internet: URL:https://arxiv.org/pdf/1412.6980v8.pdf [retrieved on Nov. 15, 2017]. cited by applicant

Agrawal, et al., “QUOTIENT: Two-Party Secure Neural Network Training and Prediction”, arxiv.org, Cornell University Library, 201 Olin Library Cornell University Ithaca, NY 14853, Jul. 8, 2019 (Jul. 8, 2019), XP081439054. cited by applicant

Chase, et al., “Private Collaborative Neural Network Learning”, IACR, International Association for Cryptologic Research vol. 20170808:183434 Aug. 7, 2017 (Aug. 7, 2017), pp. 1-17, XP061034593, Retrieved from the Internet: URL:http://eprint.iacr.org/2017/762.pdf [retrieved on Aug. 7, 2017]. cited by applicant

Mohassel et al., “SecureML: A System for Scalable Privacy-Preserving Machine Learning”, IEEE Symposium on Security and Privacy, 2017, pp. 19-38. cited by applicant

Wagh et al., “SecureNN: 3-Party Secure Computation for Neural Network Training”, Proceedings on Privacy Enhancing Technologies, 2019, vol. 1, 24 pages. cited by applicant

Hamada et al., “A Batch Mapping Algorithm for Secure Function Evaluation”, Proceedings of IEICE, Institute of Electronics, Information and Communication Engineers, JP, vol. J96-A, No. 4, Apr. 1, 2013, pp. 157-165, XP008184067, ISSN: 0913-5707, with computer-generated English translation. cited by applicant

Primary Examiner: Taylor; Nicholas R

Assistant Examiner: Pena-Santana; Tania M

Attorney, Agent or Firm: XSENSUS LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATION

(1) The present application is based on PCT filing PCT/JP2019/031941, filed Aug. 14, 2019, the entire contents of which are incorporated herein by reference.

TECHNICAL FIELD

(2) The present invention relates to a technique for calculating in a gradient descent method in secure computing.

BACKGROUND ART

(3) The gradient descent method is a learning algorithm that is often used in machine learning, such as deep learning and logistic regression. Conventional techniques for performing machine learning using a gradient descent method in secure computing include SecureML (NPL 1) and SecureNN (NPL 2).

(4) While the most basic gradient descent method is relatively easy to implement, problems such as tend to fall into a local solution, slow to converge, and the like, are known. In order to solve these problems, various optimization techniques to gradient descent methods have been proposed, and in particular, a technique called Adam is known to have a fast convergence.

CITATION LIST

Non Patent Literature

(5) NPL 1: Payman Mohassel and Yupeng Zhang, “SecureML: A System for Scalable Privacy-Preserving Machine Learning, “In IEEE Symposium on Security and Privacy, S P 2017, pp. 19-38, 2017. NPL 2: Sameer Wagh, Divya Gupta, and Nishanth Chandran, “SecureNN: 3-Party Secure Computation for Neural Network Training, “Proceedings on Privacy Enhancing Technologies, Vol. 1, p. 24, 2019.

SUMMARY OF THE INVENTION

Technical Problem

(6) However, processing of Adam includes a calculation of a square root and division, so processing costs in secure computing are very large. Meanwhile, in conventional techniques implemented by simple gradient descent methods, there is a problem in that overall processing time is long because the number of learning times required until convergence is large.

(7) In consideration of technical problems like those described above, an object of the present invention is to provide a technique that can perform calculation of a gradient descent method in secure computing at high speed while maintaining accuracy.

Means for Solving the Problem

(8) In order to solve the above problem, a secure gradient descent computation method of a first aspect of the present invention is a secure gradient descent computation method performed by a secure gradient descent computation system including a plurality of secure computation apparatuses, the secure gradient descent computation method calculating a gradient descent method while at least a gradient G and a parameter W are kept concealed, wherein β.sub.1, β.sub.2, η, and ε are predetermined hyperparameters, ◯ is a product for each element, t is the number of learning times, [G] is a concealed value of the gradient G, [W] is a concealed value of the parameter W, [M], [M{circumflex over ( )}], [V], [V{circumflex over ( )}], and [G{circumflex over ( )}]

are concealed values for matrices M, M{circumflex over ( )}, V, V{circumflex over ( )}, and G{circumflex over ( )} having the same number of elements as the gradient G, and β{circumflex over ( )}.sub.1, t, β{circumflex over ( )}.sub.2, t, and g{circumflex over ( )} are given by following equations,

(9) $\frac{1}{1-\beta_1^t} = \hat{\ }_{1,t}, \frac{1}{1-\beta_2^t} = \hat{\ }_{2,t}, \frac{}{\sqrt{\hat{v}}+} = \hat{g}$ [Math . 7]

(10) Adam is a function that calculates a secure batch mapping to output the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )} of the value g{circumflex over ( )} with the concealed value [V{circumflex over ( )}] of the matrix V{circumflex over ( )} of a value v{circumflex over ( )} as an input, a parameter update unit of each of the secure computation apparatuses calculates [M]←β.sub.1 [M]+(1−β.sub.1) [G], the parameter update unit calculates [V]←β.sub.2 [V]+(1−β.sub.2) [G]◯[G], the parameter update unit calculates [M{circumflex over ( )}]←β{circumflex over ( )}.sub.1, t [M], the parameter update unit calculates [V{circumflex over ( )}] β{circumflex over ( )}.sub.2, t [V], the parameter update unit calculates [G{circumflex over ( )}]←Adam ([V{circumflex over ( )}]), the parameter update unit calculates [G{circumflex over ( )}]←[G{circumflex over ( )}]◯[M{circumflex over ( )}], and the parameter update unit calculates [W]←[W]−[G{circumflex over ( )}].

(11) In order to solve the above problem, a secure deep learning method of a second aspect of the present invention is a secure deep learning method performed by a secure deep learning system including a plurality of secure computation apparatuses, the secure deep learning method learning a deep neural network while at least a feature X of learning data, and true data T and a parameter W of the learning data are kept concealed, wherein β.sub.1, β.sub.2, η, and ε are predetermined hyperparameters, is a product of matrices, ◯ is a product for each element, t is the number of learning times, [G] is a concealed value of a gradient G, [W] is a concealed value of the parameter W, [X] is a concealed value of the feature X of the learning data, [T] is a concealed value of a true label T of the learning data, [M], [M{circumflex over ( )}], [V], [V{circumflex over ( )}], [G{circumflex over ( )}], [U], [Y], and [Z] are concealed values of matrices M, M{circumflex over ( )}, V, V{circumflex over ( )}, G{circumflex over ( )}, U, Y, and Z having the same number of elements as the gradient G, and β{circumflex over ( )}.sub.1, t, β{circumflex over ( )}.sub.2, t, and g{circumflex over ( )} are given by following equations,

(12) $\frac{1}{1-\beta_1^t} = \hat{\ }_{1,t}, \frac{1}{1-\beta_2^t} = \hat{\ }_{2,t}, \frac{}{\sqrt{\hat{v}}+} = \hat{g}$ [Math . 8]

(13) Adam is a function that calculates a secure batch mapping to output the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )} of the value g{circumflex over ( )} with the concealed value [V{circumflex over ( )}] of the matrix V{circumflex over ( )} of a value v{circumflex over ( )} as an input, rshift is an arithmetic right shift, m is the number of pieces of learning data used for one learning, and H′ is defined by a following equation,

$H' = -\lfloor \log_2 m \rfloor$ [Math. 9]

(14) n is the number of hidden layers of the deep neural network, Activation is an activation function of the hidden layers, Activation2 is an activation function of an output layer of the deep neural network, Activation2′ is a loss function corresponding to the activation function Activation2, Activation′ is a derivative of the activation function Activation, a forward propagation calculation unit of each of the secure computation apparatuses calculates [U.sup.1]←[W.sup.0].square-solid.[X], the forward propagation calculation unit calculates [Y.sup.1]←Activation ([U.sup.1]), the forward propagation calculation unit calculates [U.sup.i+1]←[W.sup.i].square-solid.[Y.sup.i] for each i greater than or equal to 1 and less than or equal to n−1, the forward propagation calculation unit calculates [Y.sup.i+1]←Activation ([U.sup.i+1]) for each i greater than or equal to 1 and less than or equal to n−1, the forward propagation calculation unit calculates [U.sup.n+1]←[W.sup.n].square-solid.[Y.sup.n], the forward propagation calculation unit calculates

[Y.sup.n+1]←Activation2 ([U.sup.n+1]), a back propagation calculation unit of each of the secure computation apparatuses calculates [Z.sup.n+1]←Activation2′ ([Y.sup.n+1], [T]), the back propagation calculation unit calculates [Z.sup.n]←Activation′ ([U.sup.n])◯([Z.sup.n+1].square-solid.[W.sup.n]), the back propagation calculation unit calculates [Z.sup.n−i]←Activation′ ([U.sup.n−i])◯([Z.sup.n−i+1].square-solid.[W.sup.n−i]) for each i greater than or equal to 1 and less than or equal to n−1, a gradient calculation unit of each of the secure computation apparatuses calculates [G.sup.0]←[Z.sup.1].square-solid.[X], the gradient calculation unit calculates [G.sup.i]←[Z.sup.i+1].square-solid.[Y.sup.i] for each i greater than or equal to 1 and less than or equal to n−1, the gradient calculation unit calculates [G.sup.n]←[Z.sup.n+1].square-solid.[Y.sup.n], a parameter update unit of each of the secure computation apparatuses calculates [G.sup.0]←rshift ([G.sup.0], H′), the parameter update unit calculates [G]←rshift ([G.sup.i], H′) for each i greater than or equal to 1 and less than or equal to n−1, the parameter update unit calculates [G.sup.n]←rshift ([G.sup.n], H′), and the parameter update unit learns a parameter [W.sup.i] between an i layer and an i+1 layer using a gradient [G.sup.i] between the i layer and the i+1 layer, in accordance with the secure gradient descent computation method according to the first aspect, for each i greater than or equal to 0 and less than or equal to n.

Effects of the Invention

(15) According to the present invention, the calculation of the gradient descent method in the secure computing can be performed at high speed while maintaining accuracy.

Description

BRIEF DESCRIPTION OF DRAWINGS

(1) FIG. **1** is a diagram illustrating a functional configuration of a secure gradient descent computation system.

(2) FIG. **2** is a diagram illustrating a functional configuration of a secure computation apparatus.

(3) FIG. **3** is a diagram illustrating a processing procedure of a secure gradient descent computation method.

(4) FIG. **4** is a diagram illustrating a processing procedure of a secure gradient descent computation method.

(5) FIG. **5** is a diagram illustrating a functional configuration of a secure deep learning system.

(6) FIG. **6** is a diagram illustrating a functional configuration of a secure computation apparatus.

(7) FIG. **7** is a diagram illustrating a processing procedure of a secure deep learning method.

(8) FIG. **8** is a diagram illustrating a functional configuration of a computer.

DESCRIPTION OF EMBODIMENTS

(9) First, a notation method and definitions of terms herein will be described.

(10) Notation Method

(11) In the following description, symbols “—” and “A” used in the text should originally be written directly above characters immediately before them, but are written immediately after the characters due to a limitation of text notation. In mathematical formulas, these symbols are written in their original positions, that is, directly above characters. For example, “a{right arrow over ( )}” and “a{circumflex over ( )}” is expressed by the following expression in a mathematical formula:

[Math. 10]

{right arrow over (a)},â

(12) _ (underscore) in the appendix represents the subscript. For example, x.sup.y_z represents y.sub.z is the superscript to x, and x.sub.y_z represents y.sub.z is the subscript to x.

(13) A vector is written as a{right arrow over ( )}:=(a.sub.0, . . . , a.sub.n−1). a being defined by b is written as a:=b. An internal product of two vectors a{right arrow over ( )} and b{right arrow

over ( )} including the same number of elements is written as a{right arrow over ( )}.square-solid.b{right arrow over ( )}. A product of two matrices is written as (.square-solid.), and a product for each element of the two matrices or vectors is written as (◯). Those with which no operator is written represent scalar times.

(14) [a] represents encrypted a that is encrypted by using secret sharing or the like, and is referred to as a “share”.

(15) Secure Batch Mapping

(16) A secure batch mappping is a function of calculating a lookup table, which is a technique that can arbitrarily define the domain of definition and range of values. The secure batch mapping performs processing in a vector unit, so the secure batch mapping has a property that it is effective in performing the same processing on a plurality of inputs. The following illustrates specific processing of the secure batch mapping.

(17) The secure batch mapping, with the share column [a{right arrow over ( )}]:=([a.sub.0], . . . , [a.sub.m−1]), the domain of definition (x.sub.0, . . . , x.sub.l−1), and the range of values (y.sub.0, . . . , y.sub.l−1) as inputs, outputs a share with each input value mapped, specifically, a share column ([b.sub.0], . . . , [b.sub.m−1]) such that x.sub.j≤a.sub.i≤x.sub.j+1 and b.sub.i=y.sub.j for 0≤i<m. See Reference Literature 1 for the details of the secure batch mapping.

(18) Reference Literature 1: Koki Hamada, Dai Ikarashi, Koji Chida, “A Batch Mapping Algorithm for Secure Function Evaluation,” IEICE Transactions A, Vol. 96, No. 4, pp. 157-165, 2013.

(19) Arithmetic Right Shift

(20) With a share column [a{right arrow over ( )}]:=([a.sub.0], . . . , [a.sub.m−1]) and a public value t as inputs, [b{right arrow over ( )}]:=([b.sub.0], . . . , [b.sub.m−1]) in which each element of [a{right arrow over ( )}] is arithmetic right shifted by t bits is output. Hereinafter, the right shift is represented as rshift. The arithmetic right shift is a shift that performs padding on the left side by code bits rather than 0, and uses a logical right shift rlshift to configure rshift ([A*2.sup.n], n−m)=[A*2.sup.m], as in Equations (1) to (3). Note that the details of the logical right shift rlshift are referenced in Reference Literature 2.

[Math. 11]

$$[A'\times 2.\mathrm{sup.n}]=[A\times 2.\mathrm{sup.n}]+a\times 2.\mathrm{sup.n}(a\geq|A|) \quad (1)$$

$$[A'\times 2.\mathrm{sup.m}]=\mathrm{rlshift}([A'\times 2.\mathrm{sup.n}],n-m) \quad (2)$$

$$[A\times 2.\mathrm{sup.m}]=[A'\times 2.\mathrm{sup.m}]-a\times 2.\mathrm{sup.m} \quad (3)$$

(21) Reference Literature 2: Ibuki Mishina, Dai Ikarashi, Koki Hamada, Ryo Kikuchi, “Designs and Implementations of Efficient and Accurate Secret Logistic Regression,” In CSS, 2018.

(22) Optimization Technique Adam

(23) In a simple gradient descent method, the calculated gradient g is processed with w=w−ηg (η is a learning rate) to update the parameter w. Meanwhile, in Adam, processing of Equations (4) to (8) is performed to the gradient to update the parameter. Processing to calculate the gradient g is the same process even in the case of a simple gradient descent method and in the case of applying Adam. Note that t is a variable representing what number the learning is, and g.sub.t represents the gradient of the t-th time. m, v, m{circumflex over ( )}, and v{circumflex over ( )} are matrices of the same size as g, and are all initialized at 0. .square-solid..sup.t (the upper appendix t) represents the t-power.

(24) [Math. 12]

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t \quad (4) \qquad v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t \circ g_t \quad (5)$$

$$\hat{m}_{t+1} = \frac{1}{1-\beta_1^{t}} m_{t+1} \quad (6) \qquad \hat{v}_{t+1} = \frac{1}{1-\beta_2^{t}} v_{t+1} \quad (7) \qquad w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}}+\varepsilon} \circ \hat{m}_{t+1} \quad (8)$$

(25) Here β.sub.1, β.sub.2 is a constant close to 1, η is the learning rate, and ε is a value for preventing Equation (8) from being unable to calculate in the case of √v{circumflex over ( )}.sub.t+1=0. In the proposal article of Adam (Reference Literature 3), β.sub.1=0.9,

β.sub.2=0.999, η=0.001, and ε=10.sup.−8.
(26) Reference Literature 3: Diederik P Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization,” arXiv preprint arXiv: 1412.6980, 2014.
(27) In Adam, processing increases as compared to a simple gradient descent method, so the processing time required for a single learning increases. Meanwhile, the number of learning times required to converge decreases significantly, so the overall processing time required for learning is shortened.
(28) Hereinafter, an embodiment of the present invention will be described in detail. Further, the same reference numerals are given to constituent elements having the same functions in the drawings, and repeated description will be omitted.
First Embodiment
(29) In a first embodiment, the optimization technique Adam of a gradient descent method is implemented using a secure batch mapping while keeping a gradient, a parameter, and values during calculation concealed.
(30) In the following description, β.sub.1, t, β.sub.2, t, g{circumflex over ( )} are defined by the following equations.
(31) $\frac{1}{1-\beta_1^t} = \hat{\beta}_{1,t}$, [Math. 13] $\frac{1}{1-\beta_2^t} = \hat{\beta}_{2,t}$, $\frac{}{\sqrt{\hat{v}}+} = \hat{g}$
(32) β{circumflex over ( )}.sub.1, t and β.sub.2, t are calculated in advance for each t. The calculation of g{circumflex over ( )} is achieved by using a secure batch mapping that uses v{circumflex over ( )} as an input and outputs η/(√v{circumflex over ( )}+ε). The secure batch mapping is labeled Adam (v{circumflex over ( )}). Constants β.sub.1, β.sub.2, η, and ε are plaintext. Because the calculation of g includes square root and division, the processing cost in the secure computing is large. However, using a secure batch mapping is efficient because it only requires a single process.
(33) With reference to FIG. **1**, an example of a configuration of a secure gradient descent computation system of the first embodiment will be described. The secure gradient descent computation system **100** includes N (≥2) secure computation apparatuses **1**.sub.1, . . . , **1**.sub.N, as illustrated in FIG. **1**, for example. In the present embodiment, each of the secure computation apparatuses **1**.sub.1, . . . , **1**.sub.N is connected to a communication network **9**. The communication network **9** is a circuit-switched or packet-switched communication network configured to enable connected apparatuses to communicate with each other, and, for example, the Internet, a local area network (LAN), a wide area network (WAN), or the like can be used. Note that the apparatuses need not necessarily be able to communicate online via the communication network **9**. For example, information to be input to the secure computation apparatuses **1**.sub.1, . . . , **1**.sub.N may be stored in a portable recording medium such as a magnetic tape or a USB memory, and the information may be configured to be input from the portable recording medium to the secure computation apparatuses **1**.sub.1, . . . , **1**.sub.N offline.
(34) With reference to FIG. **2**, an example of a configuration of a secure computation apparatus **1**.sub.i (i=1, . . . , N) included in the secure gradient descent computation system **100** of the first embodiment will be described. As illustrated in FIG. **2**, for example, the secure computation apparatus **1**.sub.i includes a parameter storage unit **10**, an initialization unit **11**, a gradient calculation unit **12**, and a parameter update unit **13**. The secure computation apparatus **1**.sub.i (i=1, . . . , N) performs processing of each step described below while coordinating with other secure computation apparatuses **1**.sub.i′ (i′=1, . . . , N, where i≠i′), thereby implementing the secure gradient descent computation method of the present embodiment.
(35) The secure computation apparatus **1**.sub.i is a special apparatus configured by causing, for example, a known or dedicated computer including a central processing unit (CPU), a main storage apparatus (a random access memory or RAM), and the like to read a special program. The secure computation apparatus **1**.sub.i, for example, executes each processing under control of the central

processing unit. Data input to the secure computation apparatus **1**.sub.i and data obtained in each processing are stored in the main storage apparatus, for example, and the data stored in the main storage apparatus is read by the central processing unit as needed to be used for other processing. At least a portion of processing units of the secure computation apparatus **1**.sub.i may be constituted with hardware such as an integrated circuit. Each storage unit included in the secure computation apparatus **1**.sub.i may be configured by, for example, the main storage apparatus such as the random access memory (RAM), an auxiliary storage apparatus configured with a hard disk, an optical disc, or a semiconductor memory element such as a flash memory, or middleware such as a relational database or a key-value store.

(36) With reference to FIG. **3**, a processing procedure of the secure gradient descent computation method performed by the secure gradient descent computation system **100** of the first embodiment will be described.

(37) The parameter storage unit **10** stores predetermined hyperparameters β.sub.1, β.sub.2, η, and ε. These hyperparameters may be set to the values described in Reference Literature 3, for example. The parameter storage unit **10** stores pre-calculated hyperparameters β{circumflex over ( )}.sub.1, t, and β{circumflex over ( )}.sub.2, t. Furthermore, the parameter storage unit **10** stores a secure batch mapping Adam set with a domain of definition and a range of values in advance.

(38) At step S**12**, the gradient calculation unit **12** of each secure computation apparatus **1**.sub.i calculates the concealed value [G] of the gradient G. The gradient calculation unit **12** may calculate the gradient G in a manner normally performed in processing of the subject to which a gradient descent method is applied (e.g., logistic regression, learning of neural networks, and the like). The gradient calculation unit **12** outputs the concealed value [G] of the gradient G to the parameter update unit **13**.

(39) At step S**12**, the gradient calculation unit **12** of each secure computation apparatus **1**.sub.i calculates the concealed value [G] of the gradient G. The gradient calculation unit **12** may calculate the gradient G in a manner normally performed in processing of the subject to which a gradient descent method is applied (e.g., logistic regression, learning of neural networks, and the like). The gradient calculation unit **11** outputs the concealed value [G] of the gradient G to the parameter update unit **13**.

(40) At step S**13-1**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [M]←β.sub.1 [M]+(1−β.sub.1) [G] by using the hyperparameter pi stored in the parameter storage unit **10**, and updates the concealed value [M] of the matrix M.

(41) At step S**13-2**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [V]←β.sub.2 [V]+(1−β.sub.2) [G]◯[G] by using the hyperparameter β.sub.2 stored in the parameter storage unit **10**, and updates the concealed value [V] of the matrix V.

(42) At step S**13-3**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [M{circumflex over ( )}]←β{circumflex over ( )}.sub.1, t [M] by using the hyperparameter β{circumflex over ( )}.sub.1, t stored in the parameter storage unit **10**, and generates the concealed value [M{circumflex over ( )}] of the matrix M{circumflex over ( )}. The matrix M{circumflex over ( )} is a matrix having the same number of elements as the matrix M (i.e., having the same number of elements as the gradient G).

(43) At step S**13-4**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [V{circumflex over ( )}]←β{circumflex over ( )}.sub.2, t [V] by using the hyperparameter β{circumflex over ( )}.sub.2, t stored in the parameter storage unit **10**, and generates the concealed value [V{circumflex over ( )}] of the matrix V{circumflex over ( )}. The matrix V{circumflex over ( )} is a matrix having the same number of elements as the matrix V (i.e., having the same number of elements as the gradient G).

(44) At step S**13-5**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [G{circumflex over ( )}]←Adam ([V{circumflex over ( )}]) by using the secure

batch mapping Adam, and generates the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )}. The matrix G{circumflex over ( )} is a matrix having the same number of elements as the matrix V{circumflex over ( )} (i.e., having the same number of elements as the gradient G).

(45) At step S**13-6**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [G{circumflex over ( )}]←[G{circumflex over ( )}]◯[M{circumflex over ( )}] and updates the concealed value [G{circumflex over ( )}] of the gradient G{circumflex over ( )}.

(46) At step S**13-7**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i calculates [W]←[W]−[G{circumflex over ( )}] and updates the concealed value [W] of the parameter W.

(47) The algorithm for the parameter update performed from step S**13-1** to step S**13-7** by the parameter update unit **13** of the present embodiment will be indicated in Algorithm 1.

(48) TABLE-US-00001 Algorithm 1: Secure Computation Adam Algorithm Using Secure Batch Mapping Input 1: Gradient [G] Input 2: Parameter [W] Input 3: [M], [V] initialized at 0 Input 4: Hyperparameters β.sub.1, β.sub.2, β{circumflex over ( )}.sub.1, .sub.t, β{circumflex over ( )}.sub.2, t Input 5: Number of learning times t Output 1: Updated parameter [W] Output 2: Updated [M], [V] 1: [M] ← β.sub.1 [M] + (1 − β.sub.1) [G] 2: [V] ← β.sub.2 [V] + (1 − β.sub.2) [G] ◯ [G] 3: [M{circumflex over ( )}] ← β{circumflex over ( )}.sub.1, t [M] 4: [V{circumflex over ( )}] ← β{circumflex over ( )}.sub.2, t [V] 5: [G{circumflex over ( )}] ← Adam ([V{circumflex over ( )}]) 6: [G{circumflex over ( )}] ← [G{circumflex over ( )}] ◯ [M{circumflex over ( )}] 7: [W] ← [W] − [G{circumflex over ( )}]

Modification Example 1 of First Embodiment

(49) In Modification Example 1, a method of creating a table including a domain of definition and a range of values in a case of configuring the secure batch mapping Adam used in the first embodiment is devised.

(50) V{circumflex over ( )} input to the secure batch mapping Adam is always positive. The secure batch mapping Adam is a monotonically decreasing function with a very large gradient at a portion where V{circumflex over ( )} is close to 0, and has a feature that Adam (V{circumflex over ( )}) gently approaches zero when V{circumflex over ( )} increases. Because secure computing processes with fixed point numbers in terms of processing costs, very small decimal fractions such as those handled with floating point numbers are not handled. In other words, because a very small V{circumflex over ( )} is not input, the range of the values of the output of Adam (V{circumflex over ( )}) need not be set to a large value. For example, the maximum value of Adam (V{circumflex over ( )}) may be around one in a case where each of the hyperparameters is set as in Reference Literature 3 and the accuracy below the decimal point of V{circumflex over ( )} is set to 20 bits. Because the minimum value of Adam (V{circumflex over ( )}) may be determined depending on the accuracy of Adam (V{circumflex over ( )}) required, the size of the table of mapping can be determined by determining the accuracy of the input V{circumflex over ( )} and the output Adam (V{circumflex over ( )}).

Modification Example 2 of First Embodiment

(51) In Modification Example 2, the accuracy of each variable is further set as illustrated in Table 1 in the first embodiment.

(52) TABLE-US-00002 TABLE 1 VARIABLE ACCURACY (BIT) w b.sub.w β.sub.1, β.sub.2 b.sub.β β.sup.{circumflex over ( )}.sub.1, t b.sub.β{circumflex over ( )}.sub.—.sub.1 β.sup.{circumflex over ( )}.sub.2, t b.sub.β{circumflex over ( )}.sub.—.sub.2 g.sup.{circumflex over ( )} b.sub.g{circumflex over ( )}

(53) As illustrated in FIG. **4**, the parameter update unit **13** of the present modification example performs step S**13-11** after step S**13-1**, performs step S**13-12** after step S**13-2**, and performs step S**13-13** after step S**13-6**.

(54) At step S**13-11**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i arithmetic right shifts the concealed value [M] of the matrix M by b.sub.β bits. In other words, [M] rshift ([M], b.sub.β) is calculated and the concealed value [M] of the matrix M is updated.
(55) At step S**13-12**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i arithmetic right shifts the concealed value [V] of the matrix V by b.sub.β bits. In other words, [V] rshift ([V], b.sub.β) is calculated and the concealed value [V] of the matrix V is updated.
(56) At step S**13-13**, the parameter update unit **13** of each secure computation apparatus **1**.sub.i arithmetic right shifts the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )} by b.sub.g{circumflex over ( )}+b.sub.β{circumflex over ( )}_1 bits. In other words, [G{circumflex over ( )}]←rshift ([G{circumflex over ( )}], b.sub.g{circumflex over ( )}+b.sub.β{circumflex over ( )}_1) is calculated and the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )} is updated.
(57) The algorithm for the parameter update performed from step S**13-1** to S**13-7** and from S**13-11** to S**13-13** by the parameter update unit **13** of the present modification example will be indicated in Algorithm 2.
(58) TABLE-US-00003 Algorithm 2: Secure Computation Adam Algorithm Using Secure Batch Mapping Input 1: Gradient [G] Input 2: Parameter [W] Input 3: [M], [V] initialized at 0 Input 4: Hyperparameters β.sub.1, β.sub.2, β{circumflex over ( )}.sub.1, t, β{circumflex over ( )}.sub.2, t Input 5: Number of learning times t Output 1: Updated parameter [W] Output 2: Updated [M], [V] 1: [M] ← β.sub.1 [M] + (1 − β.sub.1) [G] (accuracy: b.sub.w + b.sub.β) 2: [M] ← rshift ([M], b.sub.β) (accuracy: b.sub.w) 3: [V] ← β.sub.2 [V] + (1 − β.sub.2) [G] ◯ [G] (accuracy: 2b.sub.w + b.sub.β) 4: [V] ← rshift ([V], b.sub.β) (accuracy: 2b.sub.w) 5: [M{circumflex over ( )}] ← β{circumflex over ( )}.sub.1, t [M] (accuracy: b.sub.w + b.sub.β{circumflex over ( )}.sub.—.sub.1) 6: [V{circumflex over ( )}] ← β{circumflex over ( )}.sub.2, t [V] (accuracy: 2b.sub.w + b.sub.β{circumflex over ( )}.sub.—.sub.2) 7: [G{circumflex over ( )}] ← Adam ([V{circumflex over ( )}]) (accuracy: b.sub.g{circumflex over ( )}) 8: [G{circumflex over ( )}] ← [G{circumflex over ( )}] ◯ [M{circumflex over ( )}] (accuracy: b.sub.g{circumflex over ( )} + b.sub.w + b.sub.β{circumflex over ( )}.sub.—.sub.1) 9: [G{circumflex over ( )}] ← rshift ([G{circumflex over ( )}], b.sub.g{circumflex over ( )} + b.sub.β{circumflex over ( )}.sub.—.sub.1) (accuracy: b.sub.w) 10: [W] ← [W] − [G{circumflex over ( )}] (accuracy: b.sub.w)
(59) In the present modification example, the accuracy setting is devised as follows. Here, “accuracy” indicates the number of bits in the decimal point portion, and, for example, in a case where the variable w is set to the accurate by, bit, the actual value is w*2.sup.b_w. The range of values is different for each variable and thus accuracy may be determined depending on the range of values. For example, w is likely to be a small value and parameters are very important values in machine learning, so the accuracy of the decimal point portion may be increased. Meanwhile, because the hyperparameters β.sub.2, and the like are set to approximately 0.9 or 0.999 in Reference Literature 3, there is a low need to increase the accuracy of the decimal point portion. By devising in this manner, the overall number of bits can be suppressed as much as possible, and a secure computing with a large processing cost can be efficiently calculated.
(60) In the present modification example, the following factors are devised for the right shift. For secure computing, processing at fixed point numbers rather than floating point numbers results in high speed in terms of processing costs. Here, fixed point numbers change the decimal point positions every time of multiplication, so it is necessary to adjust the decimal point positions by the right shift. However, because the right shift in the secure computing is a large cost processing, the number of right shifts may be reduced as much as possible. Because the secure batch mapping has the property of being able to arbitrarily set the range of values and the domain of definition. it is possible to adjust the number of digits as in the right shift. From such characteristics of the secure computing and the secure batch mapping, processing as in the present modification example may be more efficient.

Second Embodiment

(61) In a second embodiment, the deep learning is performed by the optimization technique Adam implemented using the secure batch mapping. In this example, learning data, a learning label, and parameters are kept concealed to perform the deep learning. Anything may be used for an activation function used in a hidden layer and an output layer, and a shape of a model of a neural network is arbitrary. Here, it is assumed to learn a deep neural network with the number of hidden layers being n layers. In other words, in a case where L is the layer number, the input layer is L=0, and the output layer is L=n+1. According to the second embodiment, a favorable learning result can be obtained even with a small number of learning times as compared to conventional techniques using a simple gradient descent method.

(62) With reference to FIG. **5**, an example of a configuration of a secure deep learning system of the second embodiment will be described. The secure deep learning system **200** includes N (≥2) secure computation apparatuses **2**.sub.i, . . . , **2**.sub.N, as illustrated in FIG. **5**, for example. In the present embodiment, each of the secure computation apparatuses **2**.sub.1, . . . , **2**.sub.N is connected to a communication network **9**. The communication network **9** is a circuit-switched or packet-switched communication network configured to enable connected apparatuses to communicate with each other, and, for example, the Internet, a local area network (LAN), a wide area network (WAN), or the like can be used. Note that the apparatuses need not necessarily be able to communicate online via the communication network **9**. For example, information to be input to the secure computation apparatuses **2**.sub.i, . . . , **2**.sub.N may be stored in a portable recording medium such as a magnetic tape or a USB memory, and the information may be configured to be input from the portable recording medium to the secure computation apparatuses **2**.sub.i, . . . , **2**.sub.N offline.

(63) With reference to FIG. **6**, an example of a configuration of a secure computation apparatus **2**.sub.i (i=1, . . . , N) included in the secure deep learning system **200** of the second embodiment will be described. As illustrated in FIG. **6**, for example, the secure computation apparatus **2**.sub.i includes a parameter storage unit **10**, an initialization unit **11**, a gradient calculation unit **12**, and a parameter update unit **13**, as in the first embodiment, and further includes a learning data storage unit **20**, a forward propagation calculation unit **21**, and a back propagation calculation unit **22**. The secure computation apparatus **2**.sub.i (i=1, . . . , N) performs processing of each step described below while coordinating with other secure computation apparatuses **2**.sub.i′ (i′=1, . . . , N, where i≠i′), thereby implementing the secure deep learning method of the present embodiment.

(64) With reference to FIG. **7**, a processing procedure of the secure deep learning method performed by the secure deep learning system **200** of the second embodiment will be described.

(65) The learning data storage unit **20** stores a concealed value [X] of a feature X of the learning data and a concealed value [T] of a true label T of the learning data.

(66) At step S**11**, the initialization unit **11** of each secure computation apparatus **2**.sub.i initializes a concealed value [W]:=([W.sup.0], . . . , [W.sup.n]) of a parameter W. The method of initializing the parameter is selected depending on an activation function, and the like. For example, it is known that in a case where an ReLU function is used for the activation function of an intermediate layer, a favorable learning result can be obtained by using the initialization method described in Reference Literature 4.

(67) Reference Literature 4: Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” In Proceedings of the IEEE international conference on computer vision, pp. 1026-1034, 2015.

(68) At step S**21**, the forward propagation calculation unit **21** of each secure computation apparatus **2**.sub.i calculates forward propagation by using the concealed value [X] of the feature of the learning data, and determines a concealed value [Y] of an output of each layer ([Y]:=[Y.sup.1], . . . , [Y.sup.n+1]). Specifically, the forward propagation calculation unit **21** calculates [U.sup.1]←[W.sup.0].square-solid.[X], [Y.sup.1]←Activation ([U.sup.1]), calculates

[U.sup.i+1]←[W.sup.i].square-solid.[Y.sup.i], [Y.sup.i+1]←Activation ([U.sup.i+1]) for each integer i greater than or equal to 1 and less than or equal to n−1, and calculates [U.sup.n+1]←[W.sup.n].square-solid.[Y.sup.n], [Y.sup.n+1]←Activation2 ([U.sup.n+1]). Here, Activation represents an activation function of any hidden layer, and Activation2 represents an activation function of any output layer.

(69) At step S**22**, the back propagation calculation unit **22** of each secure computation apparatus **2**.sub.i calculates the back propagation using the concealed value [T] of the true label of the learning data, and determines the concealed value [Z]:=([Z.sup.1], . . . , [Z.sup.n+1]) of the error of each layer. Specifically, the back propagation calculation unit **22** calculate [Z.sup.n+1]←Activation2′ ([Y.sup.n+1], [T]), [Z.sup.n]←Activation′ ([U.sup.n])◯([Z.sup.n+1].square-solid.[W.sup.n]) and calculates [Z.sup.n−i]←Activation′ ([U.sup.n−i])◯([Z.sup.n−i+1].square-solid.[W.sup.n−i]) for each integer i greater than or equal to 1 and less than or equal to n−1. Here, Activation′ represents the derivative of the activation function Activation, and Activation2′ represents a loss function corresponding to the activation function Activation2.

(70) At step S**12**, the gradient calculation unit **12** of each secure computation apparatus **2**.sub.i calculates the concealed value [G]:=([G.sup.0], . . . , [G.sup.n]) of the gradient of each layer by using the concealed value [X] of the feature of the learning data, the concealed value [Z] for the error of each layer, and the concealed value [Y] of the output of each layer. Specifically, the gradient calculation unit **12** calculates [G.sup.0]←[Z.sup.1].square-solid.[X], calculates [G.sup.i]←[Z.sup.i+1].square-solid.[Y.sup.i] for each integer i greater than or equal to 1 and less than or equal to n−1, and calculates [G.sup.n]←[Z.sup.n+1].square-solid.[Y.sup.n].

(71) At step S**13**, the parameter update unit **13** of each secure computation apparatus **2**.sub.i shifts the concealed value [G] of the gradient of each layer by the right shift by the shift amount H′, and then updates the concealed value [W]:=([W.sup.0], . . . , [W.sup.n]) of the parameter of each layer according to the secure gradient descent computation method of the first embodiment. Specifically, the parameter update unit **13** first calculates [G.sup.0]←rshift ([G.sup.0], H′), calculates [G.sup.i]←rshift ([G.sup.i], H′) for each integer i greater than or equal to 1 and less than or equal to n−1, and calculates [G.sup.n]←rshift ([G.sup.n], H′). Next, for each integer i greater than or equal to 0 and less than or equal to n, the parameter update unit **13** calculates [M.sup.i]←β.sub.1 [M.sup.i]+(1−[G.sup.i], [V.sup.i] β.sub.2 [V.sup.i]+(1−β.sub.2) [G.sup.i]◯[G.sup.i], [M{circumflex over ( )}.sup.i]←β{circumflex over ( )}.sub.1, t [M.sup.i], [V{circumflex over ( )}.sup.i]←β{circumflex over ( )}.sub.2, t [V.sup.i], [G{circumflex over ( )}.sup.i]←Adam ([V{circumflex over ( )}.sup.i]), [G{circumflex over ( )}.sup.i]←[G{circumflex over ( )}.sup.i]◯[M{circumflex over ( )}.sup.i], [W.sup.i]←[W.sup.i]−[G{circumflex over ( )}.sup.i].

(72) The algorithm of the deep learning by Adam using the secure batch mapping performed by the secure deep learning system **200** of the present embodiment will be illustrated in Algorithm 3.

(73) TABLE-US-00004 Algorithm 3: Deep Learning Algorithm by Adam Using Secure Batch Mapping Input 1: Feature [X] of learning data Input 2: True label [T] of learning data Input 3: Parameter [W.sup.1] between 1 layer and 1 + 1 Output: Updated parameter [W.sup.1] 1: Initialize all [W] 2: (1) Calculation of forward propagation 3: [U.sup.1] ← [W.sup.0] .square-solid. [X] 4: [Y.sup.1] ← Activation ([U.sup.1]) 5: for i = 1 to n − 1 do 6: [U.sup.i+1] ← [W.sup.i] .square-solid. [Y.sup.i] 7: [Y.sup.i+1] ← Activation ([U.sup.i+1]) 8: end for 9: [U.sup.n+1] ← [W.sup.n] .square-solid. [Y.sup.n] 10: [Y.sup.n+1] Activation2 ([U.sup.n+1]) 11: (2) Calculation of back propagation 12: [Z.sup.n+1] Activation2′ ([Y.sup.n+1], [T]) 13: [Z.sup.n] ← Activation′ ([U.sup.n]) ◯ ([Z.sup.n+1] .square-solid. [W.sup.n]) 14: for i = 1 to n − 1 do 15: [Z.sup.n−i] ← Activation′ ([U.sup.n−i]) ◯ ([Z.sup.n−i+1] .square-solid. [W.sup.n−i]) 16: end for 17: (3) Calculation of gradient 18: [G.sup.0] ← [Z.sup.1] .square-solid. [X] 19: for i = 1 to n − 1 do 20: [G.sup.1] ← [Z.sup.i+1] .square-solid. [Y.sup.1] 21: end for 22: [G.sup.n] ← [Z.sup.n+1] .square-solid.

[Y.sup.n] 23: (4) Update of parameter 24: [G.sup.0] ← rshift ([G.sup.0], H′) 25: for i = 1 to n − 1 do 26: [G.sup.i] ← rshift ([G.sup.i], H′) 27: end for 28: [G.sup.n] ← rshift ([G.sup.n], H′) 29: for i = 0 to n do 30: [M.sup.1] ← β.sub.1 [M.sup.i] + (1 − β.sub.1) [G.sup.i] 31: [V.sup.1] ← β.sub.2 [V.sup.i] + (1 − β.sub.2) [G.sup.i] ◯ [G.sup.i] 32: [M{circumflex over ( )}.sup.i] ← β{circumflex over ( )}.sub.1, t [M.sup.i] 33: [V{circumflex over ( )}.sup.i] ← β{circumflex over ( )}.sub.2, t [V.sup.i] 34: [G{circumflex over ( )}.sup.i] ← Adam ([V{circumflex over ( )}.sup.i]) 35: [G{circumflex over ( )}.sup.i] ← [G{circumflex over ( )}.sup.i] ◯ [M{circumflex over ( )}.sup.i] 36: [W.sup.i] ← [W.sup.i] − [G{circumflex over ( )}.sup.i] 37: end for

(74) In the actual deep learning, processing other than the initialization of the parameters of the procedure 1 of Algorithm 3 is performed for the preset number of learning times or until convergence, such as until the amount of change of the parameter becomes sufficiently small.

(75) (1) In the forward propagation calculation, the input layer, the hidden layer, and the output layer are calculated in this order, and (2) in the back propagation calculation, the output layer, the hidden layer, and the input layer are calculated in this order. However, because (3) the gradient calculation and (4) the parameter update can be processed in parallel for each of the layers, the efficiency of the processing can be increased by processing together.

(76) In the present embodiment, the activation function of the output layer and the hidden layer may be set as follows. The activation function used in the output layer is selected in accordance with the analysis to be performed. An identity function f(x)=x is used in the case of numerical prediction (regression analysis), a sigmoid function 1/(1+exp (−x)) is used in the case of a binary classification such as diagnosis of disease and spam determination, a softmax function softmax (u.sub.i)=exp (u.sub.i)/Σ.sub.j=1.sup.k exp (u.sub.j) is used in the case of classification problems of ternary values or more such as an image classification, and the like. A non-linear function is chosen for the activation function used in the hidden layer, and the ReLU function ReLU (u)=max (0, u) is frequently used in recent years. The ReLU function is known to provide favorable learning results even in deep networks and is frequently used in the field of deep learning.

(77) In the present embodiment, the batch size may be set as follows. In a case of calculating the gradient, processing the division by the batch size m with rshift is efficient. As such, the batch size m may be set to a value of a power of 2, and the amount of shift H′ at this time is determined by Equation (9). The batch size is the number of pieces of learning data used in a single learning.

[Math. 14]

$$H' = \lfloor \log_2 m \rfloor \qquad (9)$$

Modification Example 1 of Second Embodiment

(78) In the deep learning of the second embodiment, the accuracy of each value used for learning is set as illustrated in Table 2. w is a parameter between each layer, x is learning data, t is true data (teacher data) corresponding to each learning data. The output of the activation function of the hidden layer is processed to be the same as the accuracy of the true data. g{circumflex over ( )} is the value obtained by the calculation of the secure batch mapping Adam.

(79) TABLE-US-00005 TABLE 2 VARIABLE ACCURACY (BIT) w b.sub.w x b.sub.x t b.sub.y β.sub.1, β.sub.2 b.sub.β β.sup.{circumflex over ( )}.sub.1, t b.sub.β{circumflex over ( )}.sub.—.sub.1 β.sup.{circumflex over ( )}.sub.2, t b.sub.β{circumflex over ( )}.sub.—.sub.2 g.sup.{circumflex over ( )} b.sub.g{circumflex over ( )}

(80) The forward propagation calculation unit **21** of the present modification example calculates the concealed value [Y.sup.i+1] of the output of the i+1 layer for each integer i greater than or equal to 1 and less than or equal to n−1, and then shifts [Y.sup.i+1] by right shift by b.sub.w bits. In other words, [Y.sup.i+1] ← rshift ([Y.sup.i+1], b.sub.w) is calculated.

(81) The back propagation calculation unit **22** of the present modification example calculates the concealed value [Z.sup.n] of the error in the n layer, and then arithmetic right shifts [Z.sup.n] b.sub.y b.sub.y bits. In other words, [Z.sup.n] ← rshift ([Z.sup.n], b.sub.y) is calculated. The back propagation calculation unit **22** calculates the concealed value [Z.sup.n−i] of the error of the n−i

layer for each integer i greater than or equal to 1 and less than or equal to n−1, and then arithmetic right shifts [Z.sup.n−i] by b.sub.w bits. In other words, [Z.sup.n−i]←rshift ([Z.sup.n−i], b.sub.w) is calculated.

(82) In the parameter update unit **13** of the present modification example, the concealed value [G.sup.0] of the gradient between the input layer and first layer of the hidden layers is arithmetic right shifted by the shift amount b.sub.x+H′, the concealed value [G.sup.1], . . . , [G.sup.n−1] of the gradient between the hidden layers from the first layer to the n layer is arithmetic right shifted by the shift amount b.sub.w+b.sub.x+H′, and the concealed value [G.sup.n] of the gradient between the n layer of the hidden layers and the output layer is arithmetic right shifted by the shift amount b.sub.x+b.sub.y+H′. The concealed value [W]:=([W.sup.0], . . . , [W.sup.n]) of the parameter of each layer is updated in accordance with the secure gradient descent computation method of Modification Example 2 of the first embodiment.

(83) The algorithm of the deep learning by Adam using the secure batch mapping performed by the secure deep learning system **200** of the present modification example will be illustrated in Algorithm 4.

(84) TABLE-US-00006 Algorithm 4: Deep Learning Algorithm by Adam Using Secure Batch Mapping Input 1: Feature [X] of learning data Input 2: True label [T] of learning data Input 3: Parameter [W.sup.1] between 1 layer and 1 + 1 Output: Updated parameter [W.sup.1] 1: Initialize all [W] (accuracy: b.sub.w) 2: (1) Calculation of forward propagation 3: [U.sup.1] ← [W.sup.0] .square-solid. [X] (accuracy: b.sub.w + b.sub.x) 4: [Y.sup.1] ← ReLU ([U.sup.1]) (accuracy: b.sub.w + b.sub.x) 5: for i = 1 to n − 1 do 6: [U.sup.i+1] ← [W.sup.i] .square-solid. [Y.sup.i] (accuracy: 2b.sub.w + b.sub.x) 7: [Y.sup.i+1] ← ReLU ([U.sup.i+1]) (accuracy: 2b.sub.w + b.sub.x) 8: [Y.sup.i+1] ← rshift ([Y.sup.i+1], b.sub.w) (accuracy: b.sub.w + b.sub.x) 9: end for 10: [U.sup.n+1] ← [W.sup.n] .square-solid. [Y.sup.n] (accuracy: 2b.sub.w + b.sub.x) 11: [Y.sup.n+1] ← softmax ([U.sup.n+1]) (accuracy: b.sub.y) 12: (2) Calculation of back propagation 13: [Z.sup.n+1] ← [Y.sup.n+1] − [T] (accuracy: b.sub.y) 14: [Z.sup.n] ← ReLU′ ([U.sup.n]) ◯ ([Z.sup.n+1] .square-solid. [W.sup.n]) (accuracy: b.sub.w + b.sub.y) 15: [Z.sup.n] ← rshift ([Z.sup.n], b.sub.y) (accuracy: b.sub.w) 16: for i = 1 to n − 1 do 17: [Z.sup.n−i] ← ReLU′ ([U.sup.n−i]) ◯ ([Z.sup.n−i+1] .square-solid. [W.sup.n−i]) (accuracy: 2b.sub.w) 18: [Z.sup.n−i] ← rshift ([Z.sup.n−i], b.sub.w) (accuracy: b.sub.w) 19: end for 20: (3) Calculation of gradient 21: [G.sup.0] ← [Z.sup.1] .square-solid. [X] (accuracy: b.sub.w + b.sub.x) 22: for i = 1 to n − 1 do 23: [G.sup.i] ← [Z.sup.i+1] .square-solid. [Y.sup.i] (accuracy: 2b.sub.w + b.sub.x) 24: end for 25: [G.sup.n] ← [Z.sup.n+1] .square-solid. [Y.sup.n] (accuracy: b.sub.w + b.sub.x + b.sub.y) 26: (4) Update of parameter 27: [G.sup.0] ← rshift ([G.sup.0], b.sub.x + H′) (accuracy: b.sub.w) 28: for i = 1 to n − 1 do 29: [G.sup.i] ← rshift ([G.sup.i], b.sub.w + b.sub.x + H′) (accuracy: b.sub.w) 30: end for 31: [G.sup.n] ← rshift ([G.sup.n], b.sub.x + b.sub.y + H′) (accuracy: b.sub.w) 32: for i = 0 to n do 33: [M.sup.i] ← β.sub.1 [M.sup.i] + (1 − β.sub.1) [G.sup.i] (accuracy: b.sub.w + b.sub.β) 34: [M.sup.i] ← rshift ([M.sup.i], b.sub.β) (accuracy: b.sub.w) 35: [V.sup.i] ← β.sub.2 [V.sup.i] + (1 − β.sub.2) [G.sup.i] ◯ [G.sup.i] (accuracy: 2b.sub.w + b.sub.β) 36: [V.sup.i] ← rshift ([V.sup.i], b.sub.β) (accuracy: 2b.sub.w) 37: [M{circumflex over ( )}.sup.i] ← β{circumflex over ( )}.sub.1, t [M.sup.i] (accuracy: b.sub.w + b.sub.β{circumflex over ( )}.sub.—.sub.1) 38: [V{circumflex over ( )}.sup.i] ← β{circumflex over ( )}.sub.2, t [V.sup.i] (accuracy: 2b.sub.w + b.sub.β{circumflex over ( )}.sub.—.sub.2) 39: [G{circumflex over ( )}.sup.i] ← Adam ([V{circumflex over ( )}.sup.i]) (accuracy: b.sub.g{circumflex over ( )}) 40: [G{circumflex over ( )}.sup.i] ← [G{circumflex over ( )}.sup.i] ◯ [M{circumflex over ( )}.sup.i] (accuracy: b.sub.g{circumflex over ( )} + b.sub.w + b.sub.β{circumflex over ( )}.sub.—.sub.1) 41: [G{circumflex over ( )}.sup.i] ← rshift ([G{circumflex over ( )}.sup.i], b.sub.g{circumflex over ( )} + b.sub.β{circumflex over ( )}.sub.—.sub.1) (accuracy: b.sub.w) 42: [W.sup.i] ← [W.sup.i] − [G{circumflex over ( )}.sup.i] (accuracy: b.sub.w) 43: end for

(85) Similar to the second embodiment, the deep learning can be performed by repeating the

process other than the parameter initialization of the procedure 1 in Algorithm 4 until convergence or for a set number of learning times. The accuracy configuration and the positions to perform the right shift are devised in a similar manner as Modification Example 2 of the first embodiment.
(86) (1) In the calculation of forward propagation, in a case where the accuracy b.sub.x of the feature X is not too large (for example, eight bits are sufficient in the case of pixel values of image data), the right shift is omitted because b.sub.w+b.sub.x has room in the number of bits. (4) In the calculation of the parameter update, the division by the learning rate and the batch size is approximated by the arithmetic right shift by the H′ bits, and the calculation is performed simultaneously with the arithmetic right shift for accuracy adjustment, thereby improving the efficiency.

Points of the Invention

(87) In the present invention, processing of the optimization technique Adam is made efficiently with a single secure batch mapping by considering computations that the secure computing is not good at such as the square root and division included in the optimization technique Adam of the gradient descent method as one function. This allows learning with less times than conventional techniques that perform machine learning in the secure computing, and can reduce the overall processing time. Regardless of types of the machine learning model, this optimization technique can be applied to any model as long as it learns using a gradient descent method. For example, it can be used in various machine learning, such as neural network (deep learning), logistic regression, linear regression, and the like.
(88) As such, according to the present invention, the optimization technique Adam of the gradient descent method is implemented in the secure computing, allowing the learning of a machine learning model with high prediction performance with a smaller number of learning times in the secure computing.
(89) Although the embodiments of the present invention have been described above, a specific configuration is not limited to the embodiments, and appropriate changes in the design are, of course, included in the present invention within the scope of the present invention without departing from the gist of the present invention. The various kinds of processing described in the embodiments are not only executed in the described order in a time-series manner but may also be executed in parallel or separately as necessary or in accordance with a processing capability of the apparatus that performs the processing.
(90) Program and Recording Medium
(91) In a case in which various processing functions in each apparatus described in the foregoing embodiment are implemented by a computer, processing details of the functions that each apparatus should have are described by a program. By causing this program to be read into a storage unit **1020** of the computer illustrated in FIG. **8** and causing a control unit **1010**, an input unit **1030**, an output unit **1040**, and the like to operate, various processing functions of each of the apparatuses described above are implemented on the computer.
(92) The program in which the processing details are described can be recorded on a computer-readable recording medium. The computer-readable recording medium, for example, may be any type of medium such as a magnetic recording apparatus, an optical disc, a magneto-optical recording medium, or a semiconductor memory.
(93) In addition, the program is distributed, for example, by selling, transferring, or lending a portable recording medium such as a DVD or a CD-ROM with the program recorded on it. Further, the program may be stored in a storage apparatus of a server computer and transmitted from the server computer to another computer via a network so that the program is distributed.
(94) For example, a computer executing the program first temporarily stores the program recorded on the portable recording medium or the program transmitted from the server computer in its own storage apparatus. When executing the processing, the computer reads the program stored in its own storage apparatus and executes the processing in accordance with the read program. Further,

as another execution mode of this program, the computer may directly read the program from the portable recording medium and execute processing in accordance with the program, or, further, may sequentially execute the processing in accordance with the received program each time the program is transferred from the server computer to the computer. In addition, another configuration to execute the processing through a so-called application service provider (ASP) service in which processing functions are implemented just by issuing an instruction to execute the program and obtaining results without transmitting the program from the server computer to the computer is possible. Further, the program in this mode is assumed to include information which is provided for processing of a computer and is equivalent to a program (data or the like that has characteristics of regulating processing of the computer rather than being a direct instruction to the computer).
(95) In addition, although the apparatus is configured to execute a predetermined program on a computer in this mode, at least a part of the processing details may be implemented by hardware.

Claims

1. A secure deep learning method performed by a secure deep learning system including a plurality of secure computation apparatuses connected via a network, each secure computation apparatus performing processing while coordinating with other secure computation apparatuses using a secret sharing, the secure deep learning method learning a deep neural network while at least a feature X of learning data, and true data T and a parameter W of the learning data are kept concealed, the secure deep learning method comprising: receiving, by the plurality of secure computation apparatuses, input of data; calculating, by a forward propagation calculation circuitry of each of the secure computation apparatuses, [U.sup.1]←[W.sup.0]▪[X]; calculating, by the forward propagation calculation circuitry, [Y.sup.1]←Activation ([U.sup.1]); calculating, by the forward propagation calculation circuitry, [U.sup.i+1]←[W.sup.i]▪[Y.sup.i] for each i greater than or equal to 1 and less than or equal to n−1; calculating, by the forward propagation calculation circuitry, [Y.sup.i+1]←Activation ([U.sup.i+1]) for each i greater than or equal to 1 and less than or equal to n−1; calculating, by the forward propagation calculation circuitry, [U.sup.n+1]←[W.sup.n]▪[Y.sup.n]; calculating, by the forward propagation calculation circuitry, [Y.sup.n+1]←Activation2 ([U.sup.n+1]); calculating, by a back propagation calculation circuitry of each of the secure computation apparatuses, [Z.sup.n+1]←Activation2′ ([Y.sup.n+1], [T]); calculating, by the back propagation calculation circuitry, [Z.sup.n]←Activation′ ([U.sup.n])◯([Z.sup.n+1]▪[W.sup.n]); calculating, by the back propagation calculation circuitry, [Z.sup.n−i]←Activation′ ([U.sup.n−i]) ◯([Z.sup.n−i+1]▪[W.sup.n−i]) for each i greater than or equal to 1 and less than or equal to n−1; calculating, by a gradient calculation circuitry of each of the secure computation apparatuses, [G.sup.0]←[Z.sup.1]▪[X]; calculating, by the gradient calculation circuitry, [G.sup.i]←[Z.sup.i+1]▪[Y.sup.i] for each i greater than or equal to 1 and less than or equal to n−1; calculating, by the gradient calculation circuitry, [G.sup.n]←[Z.sup.n+1]▪[Y.sup.n]; calculating, by a parameter update circuitry of each of the secure computation apparatuses, [G.sup.0]←rshift ([G.sup.0], H′); calculating, by the parameter update circuitry, [G.sup.i]←rshift ([G.sup.i], H′) for each i greater than or equal to 1 and less than or equal to n−1; calculating, by the parameter update circuitry, [G.sup.n]←rshift ([G.sup.n], H′); learning, by the parameter update circuitry, a parameter [W.sup.i] between an i layer and an i+1 layer using a gradient [G.sup.i] between the i layer and the i+1 layer, for each i greater than or equal to 0 and less than or equal to n; and using the secure gradient descent computation method to predict an outcome based on the data, where β.sub.1, β.sub.2, η, and ε are predetermined hyperparameters, ▪ is a product of matrices, ◯ is a product for each element, t is the number of learning times, [G] is a concealed value of a gradient G, [W] is a concealed value of the parameter W, [X] is a concealed value of the feature X of the learning data, [T] is a concealed value of a true

label T of the learning data, [M], [M{circumflex over ( )}], [V], [V{circumflex over ( )}], [G{circumflex over ( )}], [U], [Y], and [Z] are concealed values of matrices M, M{circumflex over ( )}, V, V{circumflex over ( )}, G{circumflex over ( )}, U, Y, and Z having the same number of elements as the gradient G, and β{circumflex over ( )}.sub.1, t, β{circumflex over ( )}.sub.2, t, and g{circumflex over ( )} are given by following equations, Adam is a function that calculates a secure batch mapping to output the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )} of the value g{circumflex over ( )} with the concealed value [V{circumflex over ( )}] of the matrix V{circumflex over ( )} of a value v{circumflex over ( )} as an input, rshift is an arithmetic right shift, m is the number of pieces of learning data used for one learning, and H′ is defined by a following equation, and n is the number of hidden layers of the deep neural network, Activation is an activation function of the hidden layers, Activation2 is an activation function of an output layer of the deep neural network, Activation2′ is a loss function corresponding to the activation function Activation2, Activation′ is a derivative of the activation function Activation.

2. The secure deep learning method according to claim 1, wherein b.sub.w is an accuracy of w, by is an accuracy of an element of Y, b.sub.β is an accuracy of β.sub.1 and β.sub.2, b.sub.β{circumflex over ( )}_1 is an accuracy of β{circumflex over ( )}.sub.1, t, and b.sub.g{circumflex over ( )} is an accuracy of g{circumflex over ( )}, the forward propagation calculation circuitry calculates [Y.sup.i+1] ← Activation ([U.sup.i+1]) and then calculates [Y.sup.i+1] ← rshift ([Y.sup.+1], b.sub.w), the back propagation calculation circuitry calculates [Z.sup.n] ← Activation′ ([U.sup.n])◯([Z.sup.n+1]▪[W.sup.n]), and then calculates [Z.sup.n] ← rshift ([Z.sup.n], b.sub.y), each back propagation calculation circuitry calculates [Z.sup.n−i] ← Activation′ ([U.sup.n−i])◯([Z.sup.n−i+1]▪[W.sup.n−i]), and then calculates [Z.sup.n−i] ← rshift ([Z.sup.n−i], b.sub.w), and the parameter update circuitry learns the parameter [W.sup.i] between the i layer and the i+1 layer using the gradient [G.sup.i] between the i layer and the i+1 layer, in accordance with the secure gradient descent computation method, for each i greater than or equal to 0 and less than or equal to n, wherein the secure gradient descent computation method comprises: calculating, by the parameter update circuitry, [M] ← β1 [M]+(1−β1) [G] and then calculates [M] ← rshift ([M], bβ), calculating, by the parameter update circuitry, [V] ← β2 [V]+(1−β2) [G]◯[G] and then calculates [V] ← rshift ([V], bβ), and calculating, by the parameter update circuitry, [G{circumflex over ( )}] ← [G{circumflex over ( )}]◯[M{circumflex over ( )}] and then calculates [G{circumflex over ( )}] ← rshift ([G{circumflex over ( )}], bg{circumflex over ( )}+bβ{circumflex over ( )}_1).

3. A secure deep learning system comprising a plurality of secure computation apparatuses connected via a network, each secure computation apparatus performing processing while coordinating with other secure computation apparatuses using a secret sharing, the secure deep learning system learning a deep neural network while at least a feature X of learning data, and true data T and a parameter W of the learning data are kept concealed, each of the secure computation apparatuses including: a forward propagation calculation circuitry configured to calculate [U.sup.1] ← [W.sup.0]▪[X], [Y.sup.1] ← Activation ([U.sup.1]), [U.sup.i+1] ← [W.sup.i]▪[Y.sup.i] [Y.sup.i+1] ← Activation ([U.sup.i+1]) for each i greater than or equal to 1 and less than or equal to n−1, [U.sup.n+1] ← [W.sup.n]▪[Y.sup.n], and [Y.sup.n+1] ← Activation2 ([U.sup.n+1]); a back propagation calculation circuitry configured to calculate [Z.sup.n+1] ← Activation2′ ([Y.sup.n+1], [T]), [Z.sup.n] ← Activation′ ([U.sup.n])◯([Z.sup.n+1]▪[W.sup.n]), and [Z.sup.n−i] ← Activation′ ([U.sup.n−i])◯([Z.sup.n−i+1+]▪[W.sup.n−i]) for each i greater than or equal to 1 and less than or equal to n−1; a gradient calculation circuitry configured to calculate [G.sup.0] ← [Z.sup.1]▪[X], [G.sup.i] ← [Z.sup.i+1]▪[Y.sup.i] for each i greater than or equal to 1 and less than or equal to n−1, and [G.sup.n] ← [Z.sup.n+1][Y.sup.n]; and a parameter update circuitry configured to calculate [G.sup.0] ← rshift ([G.sup.0], H′), [G.sup.i] ← rshift ([G.sup.i], H′) for each i greater than or equal to 1 and less than or equal to n−1, and [G.sup.n] ← rshift ([G.sup.n], H′), and learn a parameter

[W.sup.i] between an i layer and an i+1 layer using a gradient [G.sup.i] between the i layer and the i+1 layer, equal to 0 and less than or equal to n, where β.sub.1, β.sub.2, η, and ε are predetermined hyperparameters, ▪ is a product of matrices, ◯ is a product for each element, t is the number of learning times, [G] is a concealed value of a gradient G, [W] is a concealed value of the parameter W, [X] is a concealed value of the feature X of the learning data, [T] is a concealed value of a true label T of the learning data, [M], [M{circumflex over ( )}], [V], [V{circumflex over ( )}], [G{circumflex over ( )}], [U], [Y], and [Z] are concealed values of matrices M, M{circumflex over ( )}, V, V{circumflex over ( )}, G{circumflex over ( )}, U, Y, and Z having the same number of elements as the gradient G, and β{circumflex over ( )}.sub.1, t, β{circumflex over ( )}.sub.2, t, and g{circumflex over ( )} are given by following equations, Adam is a function that calculates a secure batch mapping to output the concealed value [G{circumflex over ( )}] of the matrix G{circumflex over ( )} of the value g{circumflex over ( )} with the concealed value [V{circumflex over ( )}] of the matrix V{circumflex over ( )} of a value v{circumflex over ( )} as an input, rshift is an arithmetic right shift, m is the number of pieces of learning data used for one learning, and H′ is defined by a following equation, and n is the number of hidden layers of the deep neural network, Activation is an activation function of the hidden layers, Activation2 is an activation function of an output layer of the deep neural network, Activation2′ is a loss function corresponding to the activation function Activation2, Activation′ is a derivative of the activation function Activation, each of the secure computation apparatuses being configured to receive an input of data, and predict an outcome using the calculated gradient descent method based on the data.